



資料結構

Data Structure

Lab 08

姓名：林莉婕
學號：112AC1015

Lab01-Ex1. 01

Implement Postorder Traversal

Code

(a)

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;           // 節點的值
    TreeNode* left;      // 左子節點
    TreeNode* right;     // 右子節點
    TreeNode(int val) : value(val),
left(nullptr), right(nullptr) {} // 初始化節點
};

// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>&arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]);
        // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
            if (i < arr.size()) {
                current->left = new
TreeNode(arr[i]);
                q.push(current->left);
                // 將左子節點加入queue
                i++;
            }
            // 添加右子節點
            if (i < arr.size()) {
                current->right = new
TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入queue
            }
        }
    }
};
```

(b)

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 樹的節點
class TreeNode {
public:
    int value;           // 節點的值
    TreeNode* left;      // 左子節點
    TreeNode* right;     // 右子節點
    TreeNode(int val) : value(val),
left(nullptr), right(nullptr) {} // 初始化節點
};

// 樹結構
class BinaryTree {
public:
    TreeNode* root;
    BinaryTree() : root(nullptr) {} // 初始化樹

    // 用陣列構建二元樹
    TreeNode* buildTree(vector<int>&arr) {
        if (arr.empty()) return nullptr;

        queue<TreeNode*> q; // 儲存待處理的節點
        root = new TreeNode(arr[0]);
        // 建立根節點 (陣列第一個元素)
        q.push(root); // 將根節點加入queue

        size_t i = 1; // 陣列索引
        while (!q.empty()) {
            TreeNode* current = q.front(); // 取出隊列中的節點
            q.pop();

            // 添加左子節點
            if (i < arr.size()) {
                current->left = new
TreeNode(arr[i]);
                q.push(current->left);
                // 將左子節點加入queue
                i++;
            }
            // 添加右子節點
            if (i < arr.size()) {
                current->right = new
TreeNode(arr[i]);
                q.push(current->right); // 將右子節點加入queue
            }
        }
    }
};
```

```

        i++;
    }
}

return root;
}
// 中序遍歷
void inorderTraversal(TreeNode*
node) {
    if (node == nullptr) return;
    // 如果節點為空, 忽略

    inorderTraversal(node->left);
    // 遍歷左子樹
    cout << node->value << " ";
    // 訪問當前節點
    inorderTraversal(node->right);
    // 遍歷右子樹
}

void postorderTraversal(TreeNode*
node) {
    if (node == nullptr) return;
    // 如果節點為空, 忽略

    postorderTraversal(node->left); // 遍
    歷左子樹

    postorderTraversal(node->right); // 遍
    歷右子樹
    cout << node->value << " ";
    // 訪問當前節點
}

int findMax(TreeNode* node) {
    if (node == nullptr)
return -99999; //如果節點為空, 隨便回傳一個
超小值

    int leftMax =
findMax(node->left); //找左兒子
    int rightMax =
findMax(node->right); //找右兒子
    int currentMax = node->value;
    //找這個節點存的數字

    if (leftMax > currentMax) {
        currentMax = leftMax;
    }
    if (rightMax > currentMax) {
        currentMax = rightMax;
    }

    return currentMax;
}

};
int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹, NULL 表示空子節

```

```

        current->right = new
TreeNode(arr[i]);

q.push(current->right); // 將右子節點加
入queue

        i++;
    }

    return root;
}
// 中序遍歷
void inorderTraversal(TreeNode*
node) {
    if (node == nullptr) return;
    // 如果節點為空, 忽略

    inorderTraversal(node->left);
    // 遍歷左子樹
    cout << node->value << " ";
    // 訪問當前節點
    inorderTraversal(node->right);
    // 遍歷右子樹
}

void postorderTraversal(TreeNode*
node) {
    if (node == nullptr) return;
    // 如果節點為空, 忽略

    postorderTraversal(node->left); // 遍
    歷左子樹

    postorderTraversal(node->right); // 遍
    歷右子樹
    cout << node->value << " ";
    // 訪問當前節點
}

int findMax(TreeNode* node) {
    if (node == nullptr)
return -99999; //如果節點為空, 隨便回傳一個
超小值

    int leftMax =
findMax(node->left); //找左兒子
    int rightMax =
findMax(node->right); //找右兒子
    int currentMax = node->value;
    //找這個節點存的數字

    if (leftMax > currentMax) {
        currentMax = leftMax;
    }
    if (rightMax > currentMax) {
        currentMax = rightMax;
    }
}

```

點

```
vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

tree.buildTree(arr); // 建立樹
// 中序遍歷輸出
cout << "Inorder Traversal: ";
tree.inorderTraversal(tree.root);
cout << endl;

// 找左兒子最大值
int leftSubtreeMax =
tree.findMax(tree.root->left);
cout << "Max left subtree value: "
<< leftSubtreeMax << endl;

// 找右兒子最大值
int rightSubtreeMax =
tree.findMax(tree.root->right);
cout << "Max right subtree value: "
<< rightSubtreeMax << endl;

// cout << "Postorder Traversal: ";
//
tree.postorderTraversal(tree.root);

return 0;
}
```

```
[Running] cd "c:\Users\CNC\Downloads\"
Inorder Traversal: 4 2 5 1 6 3 7
Postorder Traversal: 4 5 2 6 7 3 1
[Done] exited with code=0 in 1.146 s
```

```
return currentMax;
}

};
int main() {
    BinaryTree tree; // 宣告二元樹

    // 輸入陣列用於構建樹, NULL 表示空子節點
    vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };

    tree.buildTree(arr); // 建立樹
    // 中序遍歷輸出
    cout << "Inorder Traversal: ";
    tree.inorderTraversal(tree.root);
    cout << endl;

    // 找左兒子最大值
    int leftSubtreeMax =
    tree.findMax(tree.root->left);
    cout << "Max left subtree value: "
    << leftSubtreeMax << endl;

    // 找右兒子最大值
    int rightSubtreeMax =
    tree.findMax(tree.root->right);
    cout << "Max right subtree value: "
    << rightSubtreeMax << endl;

    // cout << "Postorder Traversal: ";
    //
    tree.postorderTraversal(tree.root);

    return 0;
}
```

```
Inorder Traversal: 4 2 5 1 6 3 7
Max left subtree value: 5
Max right subtree value: 7
```

The image shows a C++ IDE with two panels. The left panel displays the source code for a binary tree implementation. The right panel shows the output of the program.

```
16 public:
21     TreeNode* buildTree(vector<int>& arr) {
46         return root;
47     }
48 }
49 // 中序遍歷
50 void inorderTraversal(TreeNode* node) {
51     if (node == nullptr) return; // 如果節點為空，忽略
52
53     inorderTraversal(node->left); // 遍歷左子樹
54     cout << node->value << " "; // 訪問當前節點
55     inorderTraversal(node->right); // 遍歷右子樹
56 }
57
58 void postorderTraversal(TreeNode* node) {
59     if (node == nullptr) return; // 如果節點為空，忽略
60
61     postorderTraversal(node->left); // 遍歷左子樹
62     postorderTraversal(node->right); // 遍歷右子樹
63     cout << node->value << " "; // 訪問當前節點
64 }
65
66 int findMax(TreeNode* node){
67     if(node == nullptr) return -99999; //如果節點為空，隨便回傳一個超小值
68     int leftMax = findMax(node->left); //找左兒子
69     int rightMax = findMax(node->right); //找右兒子
70     int currentMax = node->value; //找這個節點存的數字
71
72     if (leftMax > currentMax) {
73         currentMax = leftMax;
74     }
75     if (rightMax > currentMax) {
76         currentMax = rightMax;
77     }
78
79     return currentMax;
80 }
81
82 };
83
84 int main() {
85     BinaryTree tree; // 宣告二元樹
86
87     // 輸入陣列用於構建樹，NULL 表示空子節點
88     vector<int> arr = { 1, 2, 3, 4, 5, 6, 7 };
89 }
```

OUTPUT

```
[Running] cd "c:\Users\CNC\Downloads\" && g++ Example.cpp -o Example && "c:\Users\CNC\Downloads\" && .\Example
Inorder Traversal: 4 2 5 1 6 3 7
Postorder Traversal: 4 5 2 6 7 3 1
Max left subtree value: 5
Max right subtree value: 7
```

Discussion Section

1. 使用遞迴的方式。寫一個找最大的函數，透過回傳左兒子跟右兒子的方式去跑遞迴。先將當前的節點value存入currentValue裡，leftMax則是訪問左兒子的結果、rightMax也是，再去比大小。
2. 程式不會當掉，只會忽略，因為每一個function的最前端都有做保護機制，`if (node == nullptr)` 如果是指標為空就return。

Lab01-Q1. (題目編號)

Briefly describe the problem. (題目簡述)

Code

Paste your code here. Please use [Pygments](#) to highlight your code.
(請使用[Pygments](#)轉換程式碼格式後再貼上)

Result

Show your result in tables or screenshots.
(請使用表格或是截圖呈現結果。)

Discussion

Discuss and conclude your results, or answer questions.
(實驗討論、結論或回答問題。)

Lab01-Q#. (題目編號)		
Briefly describe the problem. (題目簡述)		
Code		
Paste your code here. Please use Pygments to highlight your code. (請使用Pygments轉換程式碼格式後再貼上)	Paste your code here. Please use Pygments to highlight your code. (請使用Pygments轉換程式碼格式後再貼上)	
result		
(a)	(b)	(c)
	Show your result in tables or screenshots. (請使用表格或是截圖呈現結果。)	
Discussion		
Discuss and conclude your results, or answer questions. (實驗討論、結論或回答問題。)		

[EXAMPLE] Q1. Factorial Function

The factorial function $n!$ has value 1 when $n \leq 1$ and value $n \cdot (n-1)!$ when $n > 1$.

Please write both a recursive and an iterative C function. Indicate the input, output, and give proper comments. Test your codes and log the results.

Code<iterative version>

```
#include<stdio.h>
#include <stdlib.h>
int main()
{
    int n,answer;
    char i;
    printf("Factorial function n!,n=");
    scanf("%d",&n); //輸入要查詢的 n
    if(n<=1)
    {
        printf("Answer=1"); //小於 1 時答案為 1
    }
    else //大於 1 時
    {
        answer=n; //計算 (n>12 時超出 int 儲存上限)
        for(i=(n-1);i>1;i--)
        {
            answer=answer*i;
        }
        printf("Answer=%d\n",answer);
    }
    system("pause");
    return 0;
}
```

<recursive version>

...

result

```
Factorial function n!,n=7
Answer=5040
```

Discussion